

The Best Tools for Static Code Analysis



This two-part series covers static code analysis and its importance, along with SonarQube integration with the Jenkins pipeline. The first article in the series discusses the importance of static code analysis and the top five tools that can be used for it.

In layman terms, ‘static code analysis is a method of analysing and evaluating source code related issues, without actually executing the program’. Static code analysis falls under the white box testing category, as here we have the entire source code available for the analysis of issues and bugs.

Static code analysis is done on the source code against a set of coding rules, which are defined by standards. Every organisation should define its set of coding standards, which every developer should follow while building applications and programs. Static code analysis is an important part of the quality check for the source code. So these code-related standards should be set by the development and quality assurance

teams together. The focus while creating these standards should be to detect the following types of source code issues for static code analysis:

- Security related issues
- Non-reliable coding issues
- Coding standards violations
- Maintainability issues
- Code duplications

Coding guidelines that can be referred to while creating an organisation’s standards are MISRA, OWASP, CWE, CERT, Pitfall, Groovyism, etc, depending on the language used.

Why is static code analysis important?

Static code analysis makes code secure, maintainable and reliable, and saves the time wasted on writing duplicated code.

If static code analysis is not carried out, we will get smelly code, leading to loss of money and loss of time in maintaining and testing the code. Bad quality code can cost billions of dollars and take up 30-35 per cent of the time of developers. So, code quality check (static code analysis) is a mandatory stage in an application’s life cycle to save the money and time of all the stakeholders.

When do you start static code analysis?

Static code analysis should be started early in the development phase, and at least before the software testing phase. It is also an important part of continuous integration and DevOps practices. It can be integrated with the continuous integration pipeline for execution in an automated manner, so that a developer can discover the issues early and resolve them before they make the code smelly.

Benefits of using static code analysis

There are multiple benefits of using static code analysis, if you maintain and configure standards.

Saves time for manual code review:

Can you imagine sitting and reviewing 1000 lines of code? This will consume not only time but your patience too. Hence, it’s best to check your source code automatically by using static code analysis tools. Since these tools can be configured to execute analysis in an inbuilt automation tool or framework, the process of code review speeds up. You will be able to fix issues in less time because the tool will provide you with the exact file and line where the problem is, and give the compliant way to correct the code. This will save lots of time in development.